

ROS2 Day4 과제 1 보고서

Lifecycle 적용 필요성 및 QoS 설정 분석

학번: 2026406076

이름: 정창규

제출일: 2026-09-14

목차

1. 과제 개요
2. Lifecycle Node 개념
3. Lifecycle 적용이 필요한 이유
4. Lifecycle 적용에 적합한 패키지
5. QoS 개념
6. 각 노드에 적합한 QoS 설정
7. QoS 선정 근거
8. Lifecycle과 QoS의 연계
9. 느낀 점

1. 과제 개요

이번 과제에서는 ROS2의 Lifecycle Node와 QoS에 대해 정리하고, 실제 로봇 시스템에서 왜 필요한지 알아보았다.

Lifecycle Node는 일반 Node와 다르게 Unconfigured, Inactive, Active, Finalized와 같이 상태를 나누어 관리한다. 이를 통해 노드가 준비되지 않은 상태에서 바로 동작하는 것을 막고, 필요한 설정과 자원이 준비된 뒤 실제 기능을 수행하도록 할 수 있다.

QoS는 Publisher와 Subscriber의 통신 방식을 정하는 설정이다. History, Depth, Reliability, Durability 등을 이용하여 메시지 보관 방식이나 전달 방식 등을 설정할 수 있다.

이번 보고서에서는 Lifecycle 적용 이유와 적용하기 적합한 패키지를 알아보고, 각 노드에 사용할 QoS 설정과 선정 이유를 정리하였다.

2. Lifecycle Node 개념

Lifecycle Node는 노드의 동작을 여러 상태로 나누어 관리하는 방식이다. 일반 Node는 실행되면 바로 동작하지만, Lifecycle Node는 준비와 실행 과정을 구분할 수 있다.

주요 상태는 Unconfigured, Inactive, Active, Finalized이다. Unconfigured는 아직 준비되지 않은 상태이고, Inactive는 설정은 끝났지만 실제 동작은 하지 않는 상태이다. Active에서는 Publish나 데이터 처리와 같은 실제 기능을 수행하며, Finalized는 종료 단계이다.

상태가 바뀔 때는 Callback 함수가 실행된다. configure를 수행하면 on_configure()가 실행되고, 이후 Inactive 상태가 된다. activate를 수행하면 on_activate()가 실행되면서 Active 상태로 전환된다. 반대로 deactivate와 cleanup을 이용하여 다시 이전 상태로 돌아갈 수 있다.

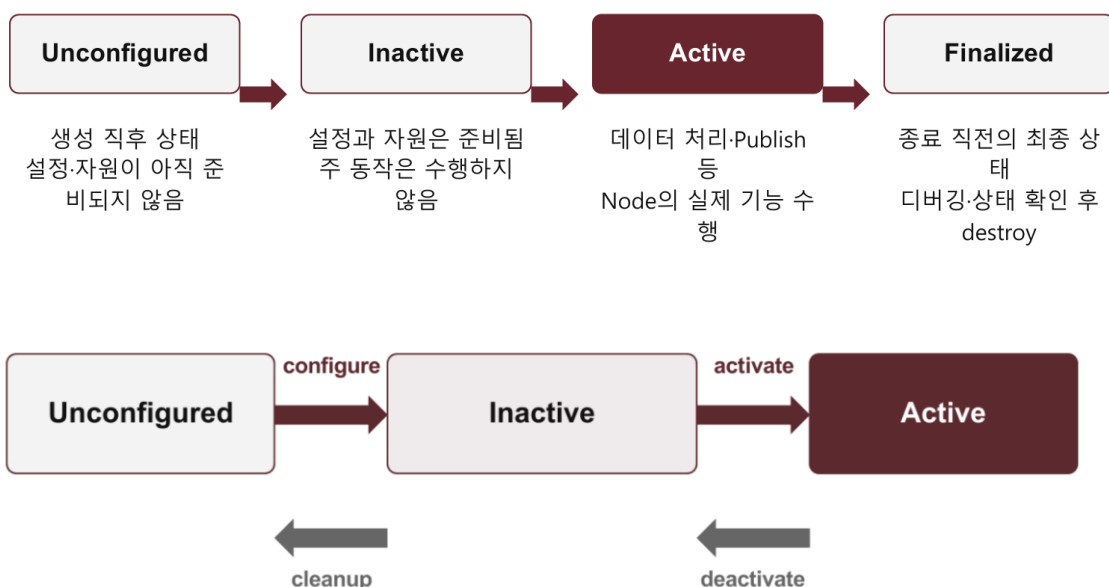


그림 1. Lifecycle Node의 주요 상태/그림 2. Lifecycle 상태 전환 과정

3. Lifecycle 적용이 필요한 이유

로봇 시스템에서는 여러 노드가 동시에 동작하기 때문에 각 노드가 언제 준비되고 언제 실제 동작을 시작하는지 관리하는 것이 중요하다.

Lifecycle을 사용하면 노드가 생성되자마자 바로 동작하지 않고, configure 과정을 통해 필요한 설정과 자원을 준비한 뒤 Active 상태에서 실제 기능을 수행하도록 할 수 있다. 이를 통해 준비되지 않은 상태에서 센서나 모터가 동작하는 상황을 줄일 수 있다.

또한 deactivate, cleanup, shutdown을 이용하여 동작을 중지하거나 자원을 정리할 수 있기 때문에 카메라, LiDAR, 모터처럼 시작과 종료 순서가 중요한 장치를 보다 안전하게 관리할 수 있다.

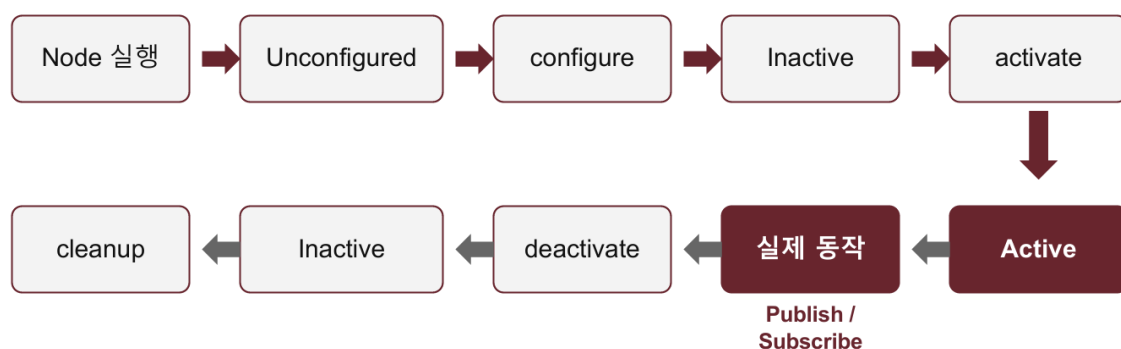


그림 3. Lifecycle Node의 실행 흐름

4. Lifecycle 적용에 적합한 패키지

Lifecycle은 실행 전에 초기화가 필요하거나, 동작 순서가 중요한 노드에 적용하기 적합하다. 특히 카메라, LiDAR, 모터처럼 하드웨어와 직접 연결되는 노드는 준비가 끝나기 전에 동작하면 문제가 생길 수 있기 때문에 Lifecycle을 사용하는 것이 유리하다.

예를 들어 센서 노드는 장치 연결과 설정을 먼저 완료한 뒤 데이터를 Publish해야 하고, 모터 제어 노드는 다른 노드의 준비 상태를 확인한 뒤 동작을 시작하는 것이 안전하다.

또한 동작 중 문제가 발생했을 때 deactivate를 이용하여 기능만 일시적으로 중지하거나, cleanup을 통해 자원을 다시 정리할 수 있어 재설정이 필요한 시스템에도 적합하다.

5. QoS 개념

QoS는 Quality of Service의 약자로, ROS2에서 Publisher와 Subscriber가 데이터를 주고받을 때 통신 특성을 정하는 설정이다. 같은 Topic과 Message Type을 사용하더라도 QoS가 서로 호환되지 않으면 메시지가 전달되지 않을 수 있다.

QoS는 데이터의 종류와 중요도에 따라 다르게 설정할 수 있다. 센서 데이터처럼 최신 값이 중요한 경우에는 일부 손실을 허용할 수 있지만, 제어 명령이나 중요한 상태 정보는 전달 여부가 더 중요할 수 있다.

주요 Policy에는 History, Depth, Reliability, Durability가 있다. History와 Depth는 메시지를 얼마나 보관할지 정하고, Reliability는 메시지 손실에 대한 처리 방식을 결정한다. Durability는 Subscriber가 늦게 연결되었을 때 이전 데이터를 받을 수 있는지와 관련된다.



그림 4. Publisher와 Subscriber 사이의 QoS 구성

6. 각 노드에 적합한 QoS 설정

ROS2에서는 모든 노드에 같은 QoS를 사용하는 것보다 전달하는 데이터의 특징에 따라 다르게 설정하는 것이 적합하다.

카메라, LiDAR와 같은 센서 노드는 데이터가 빠르게 계속 생성되기 때문에 오래된 데이터를 모두 전달하는 것보다 최신 값을 빠르게 전달하는 것이 중요하다. 따라서 Keep Last와 작은 Depth를 사용하고, 일부 데이터 손실을 허용할 수 있다면 Best Effort를 고려할 수 있다.

로봇의 상태 정보처럼 누락되면 문제가 될 수 있는 데이터를 전달하는 노드에서는 Reliable을 사용하는 것이 적합하다. 필요한 데이터의 양에 따라 History와 Depth를 함께 설정하여 메시지를 보관할 수 있다.

모터 이동 명령과 같은 제어 명령을 전달하는 노드도 Reliable을 고려할 수 있다. 다만 오래된 명령이 Queue에 계속 남아 뒤늦게 실행되지 않도록 작은 Depth나 Lifespan 설정을 함께 고려해야 한다.

또한 새로 연결된 Subscriber가 이전 상태를 바로 확인해야 하는 경우에는 Transient Local을 사용할 수 있고, 이 경우 Publisher가 보관하고 있던 최근 Sample을 늦게 참여한 Subscriber에게 전달할 수 있다.

상황	검토할 QoS 예	선택 이유
카메라 / LiDAR / 센서	Keep Last + 작은 Depth, Best Effort를 자주 고려	오래된 값보다 최신 값과 시의성이 중요
중요한 상태 정보	Reliable, 필요한 History / Depth	누락 허용도와 복구 방식에 따라 조정
제어 명령	Reliable을 고려하되 작은 Depth·Lifespan도 함께 검토	늦은 명령이 쌓여 실행되지 않도록 설계
새 Subscriber도 마지막 상태 필요	Transient Local + 호환되는 Subscriber 설정	보관 범위는 History / Depth로 결정

그림 5. 데이터 종류에 따른 QoS 설정 예시

7. QoS 선정 근거

QoS를 선정할 때에는 전달하는 데이터의 중요도, 발생 주기, 손실 허용 여부와 늦은 Subscriber의 참여 여부를 함께 고려해야 한다. Reliable은 메시지가 손실될 경우 재전송을 시도하기 때문에, 일부 데이터가 누락되어도 괜찮은 센서 정보보다 상태 정보나 제어 명령처럼 전달 여부가 중요한 데이터에 더 적합하다.

Transient Local은 Subscriber가 Publisher보다 늦게 실행되더라도 Publisher가 보관하고 있는 이전 Sample을 전달받을 수 있다는 특징이 있다. 따라서 새로운 Subscriber가 연결되었을 때 최근 상태를 바로 확인해야 하는 경우에 사용할 수 있다.

Publisher와 Subscriber에 같은 QoS 설정을 적용하여 통신 호환성도 맞추었다. 특히 Reliable Publisher와 Reliable Subscriber, Transient Local Publisher와 Transient Local Subscriber 조합은 서로 호환된다.

Reliable

전달 보장이 중요한 경우



손실 시 재전송 시도
→ 지연·트래픽이 늘 수 있음

예: 중요한 상태·명령 (요구사항 확인)

Best Effort

최신 데이터의 빠른 전달이 중요한 경우



전달을 시도하지만 재전송을 보장하지 않음
→ 일부 손실보다 시의성이 중요할 때

예: 카메라·LiDAR·고주기 센서

그림 6. Reliable과 Best Effort 비교

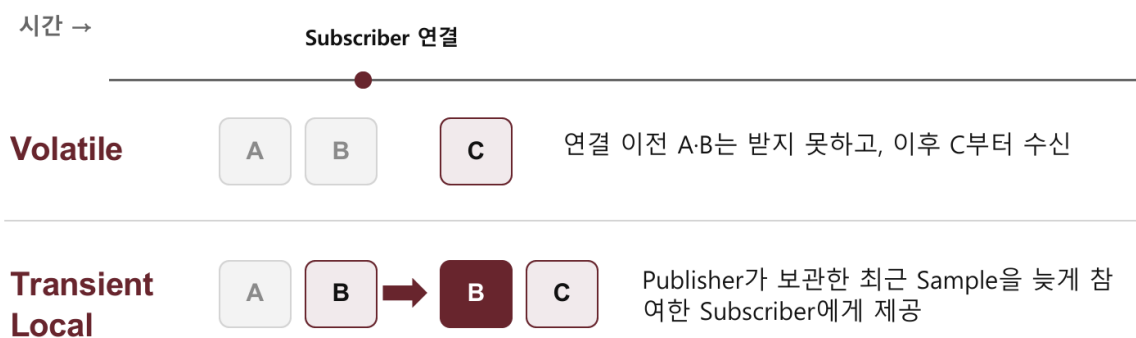


그림 7. Volatile과 Transient Local 비교

8. Lifecycle과 QoS의 연계

Lifecycle과 QoS를 함께 사용하면 노드의 상태 관리와 통신 설정을 분리해서 구성할 수 있다. 먼저 on_configure() 단계에서 QoS Profile을 결정하고 Publisher와 Subscriber를 준비한다.

설정이 끝난 뒤 노드는 Inactive 상태로 대기하고, activate 요청을 받아 Active 상태가 되면 실제 Publish와 Subscribe 동작을 수행한다. 반대로 deactivate를 수행하면 통신 동작을 중지하고 다시 Inactive 상태로 돌아간다.

이처럼 Lifecycle은 노드의 실행 시점을 관리하고, QoS는 노드 사이의 통신 방식을 결정한다. 두 기능을 함께 사용하면 준비 단계와 실제 통신 단계를 구분할 수 있어 시스템을 더 체계적으로 구성할 수 있다.

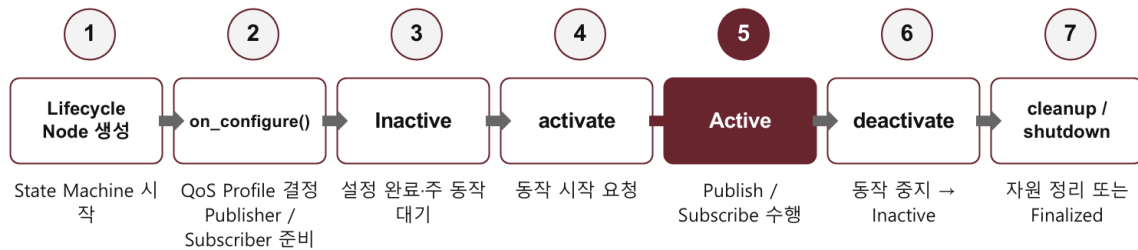


그림 8. Lifecycle과 QoS의 연계 흐름

9. 느낀 점

이번 과제를 통해 ROS2의 Lifecycle Node가 단순히 노드를 실행하고 종료하는 기능이 아니라, 노드의 상태를 단계별로 관리하기 위한 구조라는 것을 알게 되었다.

처음에는 Unconfigured, Inactive, Active 상태가 왜 나뉘어 있는지 헷갈렸지만, 각 상태에서 준비와 실제 동작을 구분한다는 점을 이해하면서 Lifecycle의 필요성을 알 수 있었다. 특히 Active 상태에서만 실제 동작을 수행하도록 구성하면 여러 장치가 함께 동작하는 로봇 시스템에서도 실행 순서를 관리하기 편리하다고 생각하였다.

또한 QoS 설정에 따라 같은 Topic을 사용하는 노드라도 통신 방식이 달라질 수 있다는 점이 새로웠다. 앞으로 ROS2 프로그램을 작성할 때 단순히 Publisher와 Subscriber를 만드는 것뿐만 아니라, 전달하는 데이터의 특징에 맞게 QoS를 설정하는 것도 중요하다고 생각하였다.